



Essential mutation framework for solving travelling salesman problem

Iyad Abu Doush¹ · Ra'ed M. Al-Khatib² · Bader Alotaibi³ · Salem Alhatamleh²

Received: 20 April 2025 / Revised: 13 July 2025 / Accepted: 29 September 2025 / Published online: 21 November 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

This study presents a novel *Essential Mutation (EM) framework* that enhances the performance of evolutionary algorithms for solving combinatorial optimization problems, with a focus on the Traveling Salesman Problem (TSP). The core innovation of proposed EM lies in its ability to preserve essential structural components, or “essential edges”, by analyzing their recurring presence in high-fitness solutions. Unlike traditional mutation operators, our proposed EM model introduces a *fitness-informed strategy* that selectively avoids mutating edges with high contribution to solution quality, effectively balancing exploration and exploitation. The importance of each edge is dynamically evaluated using population statistics and updated in each generation based on a tunable essential rate. Although the framework is algorithm-agnostic, it is integrated into a Genetic Algorithm (GA) in this work for validation. Extensive experiments on twelve TSPLIB benchmark instances compare EM against three widely used mutation operators: Swap Mutation (SM), Reverse Mutation (RM), and Center Inverse Mutation (CIM). Results show that our proposed EM model consistently achieves superior solution quality and faster convergence, particularly for larger instances. A Wilcoxon signed-rank test confirms that these improvements are statistically significant compared to RM and CIM ($p < 0.01$), while remaining competitive with SM. Although the proposed EM model incurs moderate overhead, its structural awareness enables more effective search guidance. Overall, this framework introduces a generalizable mutation strategy that improves evolutionary performance through adaptive edge preservation and is broadly applicable to other combinatorial problems.

Keywords Artificial Intelligence (AI) · Essential mutation · Evolutionary Algorithms · Traveling salesman problem · Genetic algorithm · Swap mutation · Reverse mutation

1 Introduction

The Traveling Salesman Problem (TSP) is a foundational problem in combinatorial optimization that has wide applicability in domains such as logistics, routing, and network design. It involves finding the shortest possible route that visits each city exactly once and returns to the starting point. Given its combinatorial complexity and NP-hard nature, finding optimal solutions becomes infeasible for large instances using exact methods, motivating the use of heuristic and metaheuristic approaches [1, 2].

Finding the shortest closed tour distance that benefits the salesperson who visits n cities, each of which is only visited once, and returns to the starting point, is the core of TSP. Formally, the total distance of the performed tour (f) can be written as follows if $(dist(c_i, c_j))$ represents the distance between two traveled cities and the coordinates $(c_i x, c_i y)$ of all towns (c_i) the salesman is scheduled to visit are known:

Iyad Abu Doush, Bader Alotaibi and Salem Alhatamleh contributed equally to this work.

✉ Iyad Abu Doush
iyad.abudoush@aasu.edu.kw
Ra'ed M. Al-Khatib
raed.m.alkhatib@yu.edu.jo
Bader Alotaibi
s00053306@auk.edu.kw
Salem Alhatamleh
2022781013@ses.yu.edu.jo

¹ College of Computer and Systems Engineering, Abdullah Al Salem University, Khaldiah 72100, Kuwait

² Department of Computer Science, Yarmouk University, Irbid 21163, Jordan

³ American University of Kuwait, Salmiyah 13034, Kuwait

$$f = \left(\sum_{i=1}^{n-1} \text{dist}(c_i, c_{i+1}) \right) + \text{dist}(c_n, c_1) \quad (1)$$

Notably, the TSP shares structural similarities with other classical discrete optimization problems, such as the Flexible Job Shop Scheduling Problem (FJSP) [3]. Both issues involve combinatorial search spaces and are often tackled using permutation-based encodings and neighborhood search operators. For instance, in both TSP and FJSP, candidate solutions can be represented as permutations of tasks or cities, and improved through localized modifications. Shared strategies such as crossover, mutation, and local refinement are commonly used across both problem types [4, 5]. Recent studies have explored advanced metaheuristics for FJSP, including disjunctive graph-based approaches and problem-specific evolutionary algorithms tailored to customized manufacturing contexts [3, 6]. These parallels provide a broader context for general-purpose mutation frameworks like the one proposed in this study.

Among metaheuristics, Evolutionary Algorithms (EAs) have proven to be powerful tools for solving complex optimization problems. Inspired by the process of natural evolution, EAs maintain a population of candidate solutions and evolve them through iterations of selection, recombination, and mutation [7, 8]. These operators help EAs strike a balance between exploration (diversifying the search space), and exploitation (refining high-quality solutions). The flexibility of EAs allows them to be adapted to various types of optimization landscapes—continuous, discrete, or combinatorial [9, 10].

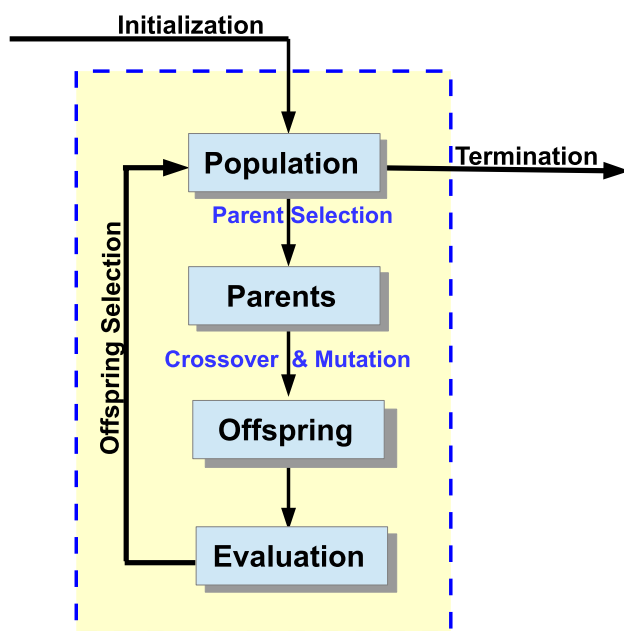


Fig. 1 Main steps for evolutionary algorithm

A prominent variant of EAs is the Genetic Algorithm (GA), which operates on a population of solutions encoded as chromosomes [11]. In each generation, individuals are selected based on fitness, combined through crossover, and modified using mutation. Mutation plays a key role in maintaining population diversity and preventing premature convergence, especially in problems like TSP, where maintaining beneficial structure is critical.

Figure 1 illustrates the general framework of EAs. The foundational variants of EAs include GA [11], Genetic Programming (GP) [12], Evolution Strategies (ES) [13], and Evolutionary Programming (EP) [14]. EAs differ in terms of their design choices, including selection mechanisms, variation operators, and solution representations. While traditional implementations of methods like GAs and ES were often associated with specific types of problem landscapes—such as binary or continuous domains—subsequent research has demonstrated that these algorithms are *highly flexible and not inherently limited to specific environments*. With appropriate encoding and the design of customized variation operators, GAs can effectively solve a wide range of problems, including grouping, permutation-based, and continuous optimization tasks [15, 16]. Other metaheuristic approaches, including Ant Colony Optimization (ACO) [17], Particle Swarm Optimization (PSO) [18], and Hybrid GAs that combine local search strategies with global search frameworks [19], have also shown strong performance in tackling TSP variants. These methods offer diverse mechanisms for balancing exploration and exploitation.

Despite the effectiveness of these algorithms, one persistent challenge is maintaining structural quality during the mutation phase. Conventional mutation operators often disrupt key solution components, leading to a loss of convergence speed or overall solution quality. Mutation plays a particularly vital role in preventing premature convergence and maintaining population diversity [4]. However, traditional mutation operators often disregard problem-specific structures, which can disrupt promising partial solutions. This observation motivates the design of structurally-aware mutation mechanisms. To address this, we propose the Essential Mutation (EM) framework, a novel mutation strategy that preserves structurally significant components—called essential edges—which frequently appear in high-quality solutions. By avoiding disruption to these edges, EM guides the search process more effectively, thereby improving the algorithm's ability to converge on better solutions.

While EM is a general-purpose framework compatible with various EAs, in this study, we implement it within a GA and evaluate its performance using three classical mutation operators: Swap Mutation, Reverse Mutation, and Center Inverse Mutation. Experiments conducted on twelve benchmark instances from TSPLIB demonstrate that EM

significantly improves both the quality and consistency of solutions compared to standard mutation strategies.

The rest of the paper is organized as follows: Section 2 describes the proposed methodology. Section 3 presents the experimental results and analysis. Section 4 concludes the paper and outlines some future directions.

2 Methodology

Mutation is a fundamental operator employed by many EAs to introduce diversity into the population of solutions. By promoting variability, mutation enhances the algorithm's ability to explore the search space effectively, ultimately improving the evolutionary process, as explained in Algorithm 1. This paper presents a novel mutation technique inspired by a recently discovered mutation bias pattern observed in the *Arabidopsis thaliana* plant [20]. The proposed technique, termed *essential mutation*, leverages these biological insights to refine mutation strategies in EAs, aiming to improve convergence and solution quality.

The step-by-step process of the proposed *essential mutation* technique is detailed in Algorithm 2, while the overall flow of GA with the essential mutation method is shown in Fig. 2.

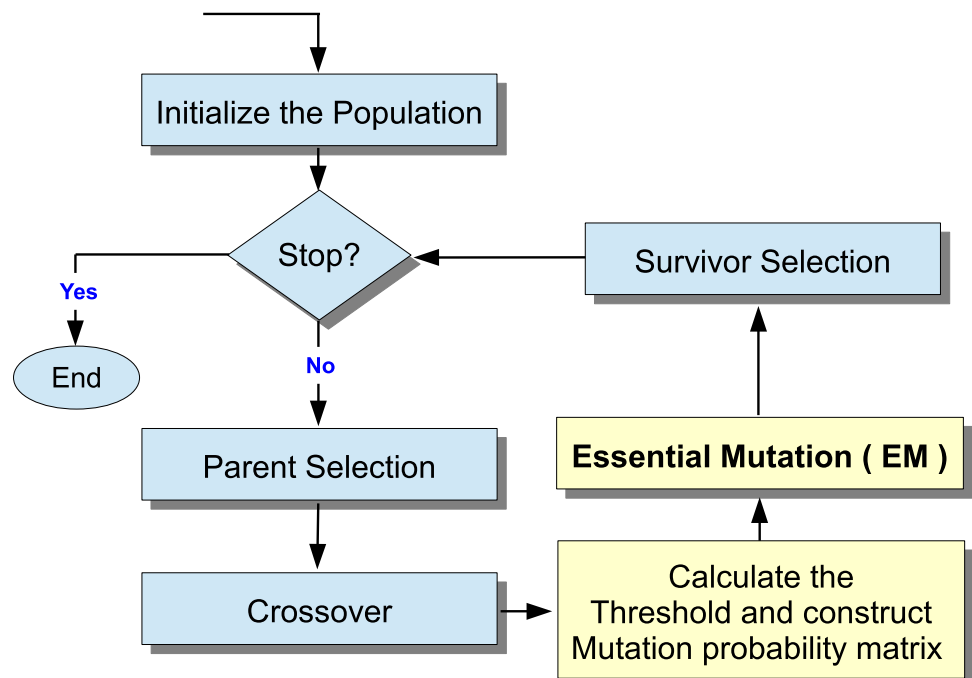
The methodology employed in this research endeavors to investigate the effectiveness of the proposed “Essential Mutation” framework in improving the performance of GAs for solving the TSP problem. This research design aims to conduct a rigorous test of the new mutation operator against other conventional mutation operators, specifically swap

mutation, reverse mutation, and center inverse mutation. The core of an essential mutation technique is the basis for developing the proposed framework to improve the convergence and solution quality of GAs for the TSP problem.

The EM operator forms part of our proposed framework, which is elaborately derived from the so-called Essential Mutation technique proposed in the recent literature [21]. The main steps for the essential mutation process to solve the TSP problem are illustrated in Fig. 3. This new mutation scheme tends to judge the importance of a route section within the context of the TSP. The proposed EM operator looks for whether any given route has an essential edge by analyzing the average fitness of all the tours that share that particular route, and then retains the essential edges during mutation. When we have the TSP solution, the algorithm will create a list of essential edges (L) (not to be muted). Then, it starts by iterating through each city in the solution and picking cities with a mutation rate percentage to swap. But these swapped cities must not be part of the list of essential edges. This process repeats until all cities have been evaluated, after which the mutated individual is returned as the new solution. This unique process works to improve convergence to optimal solutions by protecting critical routes that give significant value to the overall quality of solutions.

To elaborate on how essential mutation creates the list of essential edges in TSP, a set of proposed solutions with the best fitness is computed as seen in Fig. 4. The algorithm will then identify the routes that are repeated in these best solutions to be identified as essential edges. Therefore, the generated list of essential edges will be as shown in Fig. 5.

Fig. 2 GA with our proposed Essential Mutation (EM) method



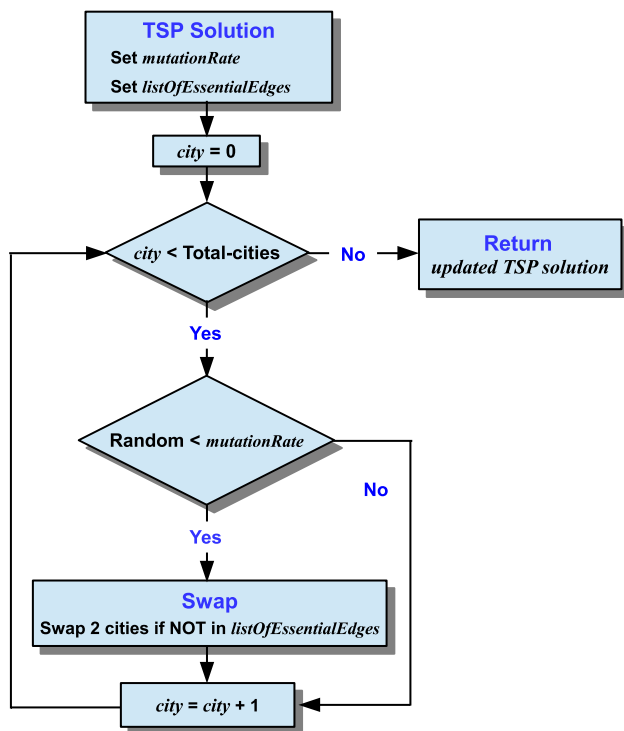


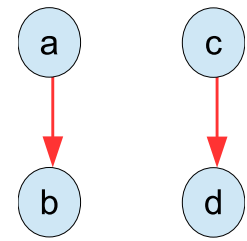
Fig. 3 Essential mutation steps for solving the TSP problem

To provide a detailed explanation of how the essential mutation algorithm generates the list of essential edges in the TSP, the process begins by evaluating a set of candidate solutions. These solutions are ranked according to their fitness, and the best performing solutions are selected for further analysis, as depicted in Fig. 4. Fitness is typically measured by the total distance of the route, where shorter distances indicate better solutions.

Once the top solutions are identified, the algorithm examines the routes within these solutions to detect recurring patterns or common connections between cities. Edges (city-to-city connections) that frequently appear in the best solutions are marked as essential edges, as they are likely to contribute to optimal or near-optimal paths. This step ensures that critical connections that enhance the quality of the solution are preserved and prioritized during the mutation process.

The final list of essential edges, generated from this analysis, is then compiled and presented in Fig. 5. By retaining these essential edges, the algorithm improves its efficiency

Fig. 5 Generated list of essential edges



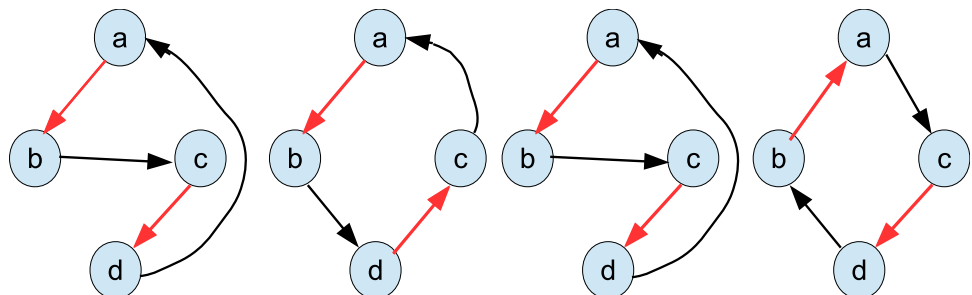
by reducing the likelihood of disrupting high-quality solutions, while still allowing for beneficial mutations to explore new, potentially superior paths. This approach balances exploration and exploitation, enhancing the overall effectiveness of the mutation process.

It is important to note that the fitness of each unique edge is accumulated based on its occurrence across different proposed solution routes. The fitness contribution of an edge is derived from the fitness of the route in which it appears. To calculate the final fitness value of each unique edge, the total accumulated fitness is divided by the frequency of the edge's occurrence across all the proposed solution routes for the TSP problem. This approach ensures that edges appearing in high-quality solutions are given greater importance while normalizing their contribution based on how frequently they appear.

To identify structurally significant components within high-quality solutions, we introduce the *Essential Evaluation* algorithm (Algorithm 1), which extracts a list of candidate essential edges based on their contributions to fitness across the population. For each individual in the population, the algorithm accumulates the total fitness contribution of each traversed edge. Moreover, it tracks the frequency of occurrence for every unique edge encountered. After processing all individuals, the algorithm computes the average fitness for each edge by dividing its total fitness by its frequency of appearance.

To select the most influential edges, the algorithm ranks all unique edges based on their average fitness and selects the top subset according to a predefined *EssentialRatio*. These high-ranking edges are returned as *essential edges*, which represent consistently strong building blocks in the population. This mechanism allows the Essential Mutation operator to focus search efforts on preserving beneficial structural components while maintaining diversity for exploration.

Fig. 4 Set of proposed solutions with best fitness



Input: *Population, EssentialRatio*
Output: a list of candidate essential edges

```

1: for each individual in Population do
2:   fitness = Fitness(individual)
3:   for i=0 to length(individual)-1 do
4:     fromCity = individual[i]
5:     toCity = individual[i+1] ▷ accumulating fitness for that edge
6:     pathTotalFitness[fromCity][toCity] += fitness
7:     \\Count the number of unique edges
8:     if pathCounter[fromCity][toCity]==0 then
9:       uniqueEdgesCounter++
10:    end if
11:    pathCounter[fromCity][toCity]++
12:  end for ▷ Repeat for the final edge connecting back
13:  fromCity = individual[length(individual)]
14:  toCity = individual[0]
15:  pathTotalFitness[fromCity][toCity] += fitness
16:  if pathCounter[fromCity][toCity]==0 then
17:    NumberOfEssentialEdges
18:    uniqueEdgesCounter++
19:  end if
20:  pathCounter[fromCity][toCity]++
21: end for ▷ Divide the total fitness of each edge by its frequency
22: pathAvgFitness = pathTotalFitness/pathCounter
23: NumberOfEssentialEdges = uniqueEdgesCounter × EssentialRatio
24: Sort pathAvgFitness while maintaining the edge position for that average fitness
25: Return: list of NumberOfEssentialEdges from sorted pathAvgFitness

```

Algorithm 1 Essential Evaluation

The *Essential Mutation* algorithm (Algorithm 2) enhances traditional mutation operations by preserving structurally important components—specifically, the essential edges identified in the population. For each city in a given TSP individual, a random number is generated to determine whether a mutation should occur, based on a predefined *mutationrate*. If a mutation is triggered, a second random city is selected for a potential swap.

However, before performing the swap, the algorithm checks whether the edge following the current city is classified as essential. If the edge is not on the list of *essential edges*, the swap is executed; otherwise, the edge is preserved to maintain high-quality structural features. This selective mutation strategy allows the algorithm to explore new solution configurations while safeguarding critical substructures that consistently contribute to fitness, thus balancing exploration and exploitation effectively.

Input: *Individual, MutationRate, listOfEssentialEdges*
Output: The mutated *individual*

```

1: \\Going over each City
2: for city = 0 to length(individual) - 1 do
3:   random = U(0,1)
4:   if random < mutationRate then
5:     random2 = U(0,1) ▷ pick city to swap
6:     swapWith = int(random2 × len(individual))
7:     \\This edge not essential
8:     if not (listOfEssentialEdges[city][city + 1]) then
9:       temp = individual[city]
10:      individual[city] = individual[swapWith]
11:      individual[swapWith] = temp
12:    end if
13:  end if
14: end for
15: Return: individual

```

Algorithm 2 Essential Mutation

The fitness of each unique edge can be mathematically represented as follows:

$$F(e_i) = \frac{\sum_{j=1}^n f(S_j) \cdot \delta(e_i, S_j)}{\sum_{j=1}^n \delta(e_i, S_j)}, \quad (2)$$

Where:

- $F(e_i)$ is the normalized fitness of the edge e_i .
- n is the total number of proposed solution routes.
- $f(S_j)$ is the fitness value of the j -th solution route.
- $\delta(e_i, S_j)$ is an indicator function defined as follows:

$$\delta(e_i, S_j) = \begin{cases} 1, & \text{if edge } e_i \text{ is present in } S_j \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

In this equation:

- The numerator accumulates the fitness values of all the routes where the edge e_i appears.
- The denominator counts how many times the edge e_i appears in all proposed solution routes, normalizing the accumulated fitness.

This formulation ensures that edges contributing to high-quality solutions are given appropriate weight, while also accounting for their frequency across different solutions.

To introduce the possibility of dynamically adapting the EM framework over evolving generations, we propose a new operator for computing the number of essential edges based on the so-called essential rate multiplied by the overall number of unique edges in the given TSP instance. The iterative update of the list of essential edges ensures that the framework remains responsive to changes in the population structure, contributing to its adaptability and effectiveness in differentiating critical edges from less influential ones.

We have tested our proposed essential mutation framework against three different mutation methods [22]:

- **Swap Mutation (SM):** swap a random city with another random city.
- **Center Inverse Mutation (CIM):** split the route into two parts, invert them, then place them back in the same position

- **Reverse Mutation (RM):** randomly select a sub-sequence within the route and reverse it.

2.1 Experimental setup and parameter tuning

In our experiment, we used the symmetric TSP test cases from the TSPLIB dataset [23]. It consists of a set of nodes such that the distance between node i and node j is the same as the distance between node j and node i .

The experiments are conducted using a standardized GA setup, incorporating the proposed EM framework alongside conventional mutation operators. To ensure a fair comparison, all mutation strategies (EM, SM, RM, and CIM) have been evaluated under the same GA configuration which are: population size = 300, elite size = 60, mutation rate = 0.05, crossover rate = 0.8, number of generations = 2000, and 30 independent runs per dataset. To ensure a fair and consistent experimental environment, we applied a manual grid search tuning to key parameters, including population size, crossover rate, mutation rate, and essential rate. This method involved running the algorithm across a range of candidate values and selecting the configuration that consistently yielded the best performance across multiple benchmark instances. The aim was to establish a balanced comparison framework without introducing bias toward any specific mutation operator. All experiments were conducted on a computing platform with standard hardware specifications, ensuring the reliability and reproducibility of the results. This consistent setup allows us to confidently attribute observed performance differences to the mutation strategies under investigation.

To assess the statistical significance of the differences in performance between the proposed EM-based model and other mutation operators, we conducted pairwise Wilcoxon signed-rank tests across the 12 TSP benchmark instances. The results are shown in Table 1.

The analysis reveals that our proposed EM model significantly outperforms both Reverse Mutation (RM) and Center Inverse Mutation (CIM), with p -values of 0.0068 and 0.0005, respectively, indicating strong statistical evidence against the null hypothesis of no difference. However, the difference between EM and Swap Mutation (SM) is not statistically significant ($p = 0.3013$), suggesting comparable performance under the tested configurations.

These results reinforce the robustness of the proposed EM strategy, particularly its advantage over mutation techniques that induce broader structural changes, such as RM and CIM, by preserving essential edge structures in high-quality solutions.

We conducted a calibration test to identify the optimal combination of Mutation Rate and Essential Rate for the algorithm. This tuning was performed using the proposed

Table 1 Wilcoxon signed-rank test p -values comparing proposed EM-based model against other mutation operators across 12 TSP benchmark instances

Comparison	p -value
EM vs. Swap Mutation (SM)	0.3013
EM vs. Reverse Mutation (RM)	0.0068
EM vs. Center Inverse Mutation (CIM)	0.0005

Table 2 Tuning of mutation rate and essential rate using the proposed EM-based framework with SM as the base operator

		MutateRate = 0.005		MutateRate = 0.01		MutateRate = 0.05	
Essential Rate		Best	Mean	Best	Mean	Best	Mean
0.05	a280	9674.16	10512.25	13226.68	13690.07	25871.95	26444.83
0.1		9603.63	10426.65	12650.01	12941.17	23850.75	24745.42
0.3		7757.91	8408.01	7665.49	8388.12	9243.55	10080.23
0.5		10323.79	10984.83	10005.31	10489.54	8567.76	9180.6
0.05	berlin52	9519.71	10297.42	9579.11	9997.3	11778.37	12802.18
0.1		9297.09	9989.48	9098.35	9921.79	9398.21	10238.6
0.3		9469.6	10339.83	9102.62	10271.84	9343.65	10256.4
0.5		10477.14	11537.9	9864.47	11022.93	10008.66	10673.47
0.05	ch150	13652.61	14243.94	14901.34	16188.62	36547.05	38239.62
0.1		12011.37	13069.9	14602.55	15681.87	30645.59	33771.69
0.3		12842.89	13654.14	12548.81	13343.86	12559.57	13255.21
0.5		16044.62	17502.79	15132.84	16573.24	14268.24	15444.97
0.05	lin318	180778.8	187766.2	242669.2	257605.5	467481.1	475018.8
0.1		175627.8	187145	235957.3	242094.2	446111.6	449850.8
0.3		130801.6	148284.5	138709.6	152278.1	192549.7	203972.3
0.5		183143.8	189987	166942.8	177265.4	157099.4	164584.7
0.05	bier127	164551.8	188990.5	192951	208616.5	415906	423677.9
0.1		176023.4	192692.4	188119.9	205840.1	339937	372525.6
0.3		188014.9	198903.1	187596.9	197911	176530.4	188365.8
0.5		217063.8	237961.5	206805	224356.2	206200.8	214241.9
0.05	eil51	527.73	577.99	533.18	573.62	684.02	726.78
0.1		526.64	565.48	541.48	568.17	516.71	560.11
0.3		488.19	572.9	533.83	583.92	530.65	586.39
0.5		594.9	684.83	592.53	635.13	493.81	610.16
0.05	kroA100	37035.84	41548.09	38679.05	43514.91	92945.89	97234.24
0.1		35078.54	41219.31	33873.79	41712.02	80000.44	81781.64
0.3		38344.56	43050.12	37944.53	41783.94	38389.16	43168.59
0.5		50456.27	53339.31	45731.23	50016.94	42996.29	45852.49

EM-based framework applied to Swap SM as the base mutation operator. Based on the results shown in Table 2, the algorithm performs better at lower *Mutation Rate* values, as indicated by the lower average cost values observed when the *Mutation Rate* is set to 0.005 and 0.01. Regarding the *Essential Rate*, the average results suggest that the optimal values fall between 0.1 and 0.3.

Additionally, we observed that when the *Mutation Rate* is increased to 0.05, a higher *Essential Rate* of 0.05 yields better performance. After analyzing the best and second-best performing combinations of *Mutation Rate* and *Essential Rate*, we concluded that the most effective combination is a *Mutation Rate* of 0.05 paired with an *Essential Rate* of 0.1. This combination consistently produced the most favorable outcomes in our test cases.

3 Results and discussion

To evaluate the effectiveness of the proposed EM-based framework, a series of comprehensive experiments was conducted using a GA as the test environment. EM was

compared against three well-known mutation strategies commonly used in TSP optimization: Swap Mutation (SM), Reverse Mutation (RM), and Center Inverse Mutation (CIM).

All experiments were executed on a PC equipped with an Intel®Core™ i9-13900K CPU (up to 5.8 GHz) and 32 GB RAM, running MATLAB R2024b on Windows 11. The GA parameters were kept constant across all tests to ensure fairness: mutation rate = 0.05, essential rate = 0.1, population size = 300, elite size = 60, and number of generations = 2000.

Figure 6 illustrates the best tour result for each mutation strategy on the st70 dataset. Figure 7 further presents the convergence behavior of each strategy over 500 generations. The results clearly indicate that both EM and SM achieved faster convergence compared to RM and CIM mutation operators. This can be attributed to their ability to refine diversity while preserving the underlying structure of promising solutions, which accelerates the search process without sacrificing quality.

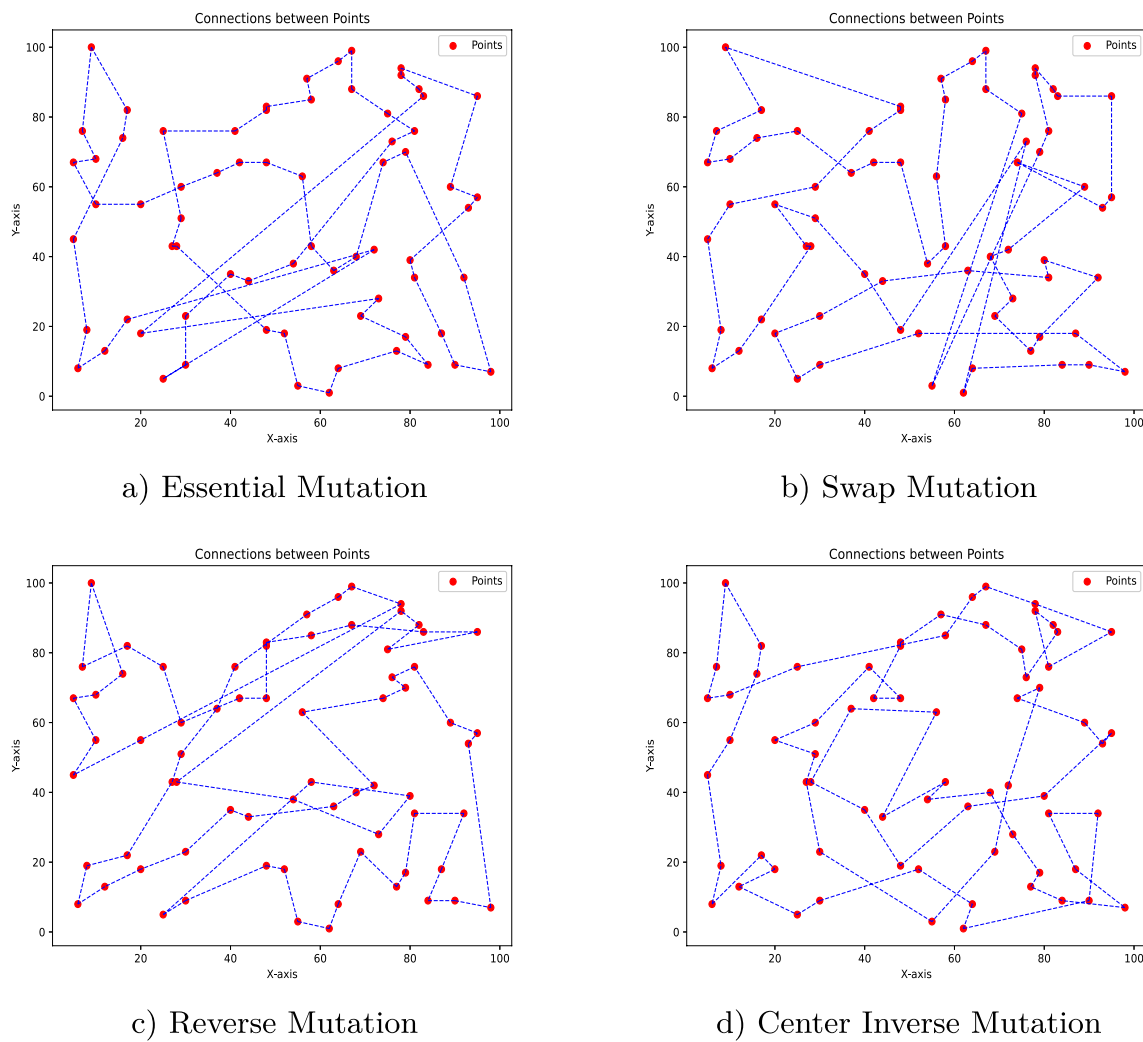


Fig. 6 Best GA Routes with Essential, Swap, Reverse, and Center Inverse Mutations when applied to Solve st70 dataset

Fig. 7 Mutations improvement over 500 generations

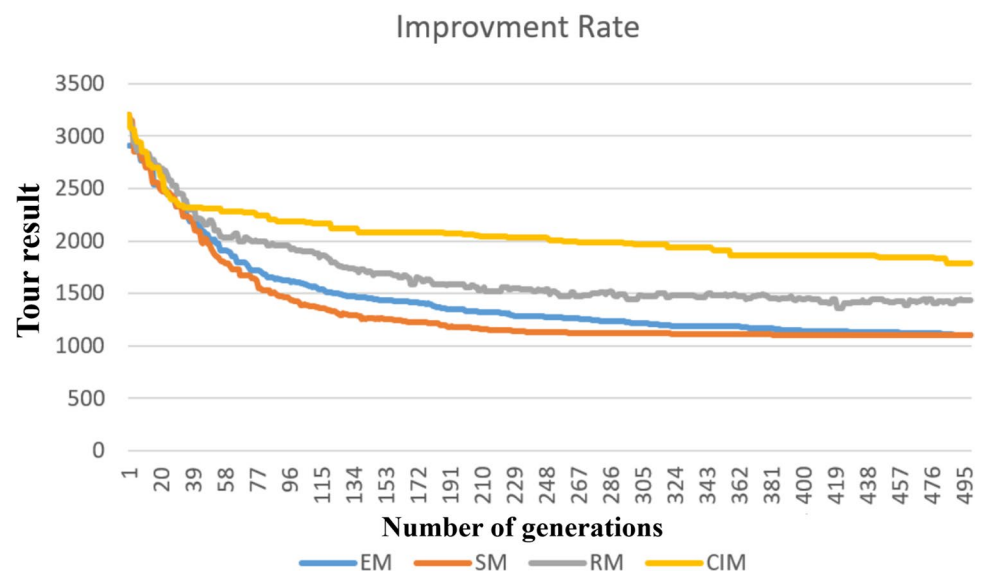


Table 3 Comparison of proposed EM against SM, RM, and CIM using the TSPLIB dataset in terms of best, average, and standard deviation of solution quality

Test	EM			SM			RM			CIM		
	Best	Avg	STDEV	Best	Avg	STDEV	Best	Avg	STDEV	Best	Avg	STDEV
a280	9305.9	10393.19	509.2893	9613.6	10598.1	375.2861	25048.4	25456.53	214.4653	16104.4	16965.23	498.2856
berlin52	8935.5	9958.227	573.646	8658.6	10185.79	815.989	8381.6	9262.127	388.0831	9208.3	10205.76	545.8238
bier127	169898.4	191603.5	12132.4	165799.1	190757.6	9966.442	377630.7	398325.2	8357.091	269269.6	293162	10317.51
ch150	11681.6	13672.24	1006.243	12033	14769.44	1002.257	34429	35860.41	448.4774	21465.7	23025.38	843.6545
eil51	512.8	574.6333	35.10944	514.4	557.0067	22.51447	486.6	539.1967	28.40889	533.2	587.9867	28.84885
kroA100	34554.4	40883.37	3417.103	33609.7	40802.1	3777.359	81901.6	87428	2254.403	52948.1	57306.66	2954.262
kroA200	84790.9	92292.83	3819.631	86242.6	95190.7	5398.451	223182.9	231600	2841.384	137275.7	148963.5	4750.758
lin318	166595.3	183976.1	6779.134	173262.7	190484.8	6589.481	448438.7	456247.8	3784.207	300300.6	312622.1	5846.958
pr152	199271	261855.8	24072.22	237586.8	271867.6	18836.94	640742.4	659313.5	7916.232	336478.2	398336.9	23073.78
pr226	408982.6	464713.2	29457.29	416184.5	454553.5	27403.35	1162368	1196522	13220.83	661441.4	732410.9	27239.2
rat195	5737	6286.047	313.8443	5922.9	6634.593	393.1045	15008.4	15417.58	162.4125	9298.9	9932.94	368.5094
st70	886.2	1078.987	81.16042	918.4	1057.783	68.18089	920.3	1068.623	71.02582	1029.9	1198.137	60.10967

3.1 Comparative analysis across datasets

To ensure the robustness and scalability of the proposed EM framework, we evaluated all four mutation strategies (EM, SM, RM, and CIM) across all used twelve benchmark TSP instances from TSPLIB, each with a known optimal solution. These instances were selected to provide a diverse range of problem sizes and complexities. For each dataset, 30 independent runs were performed to assess statistical reliability.

The evaluation process focused on two primary performance indicators:

- **Solution quality:** assessed using the best, average, and standard deviation of the tour lengths over 30 runs.
- **Convergence rate:** evaluated by tracking the evolution of the quality of the solution between generations.

Table 3 summarizes the comparative results. It illustrates the best, average, and standard deviation of solution quality for each type of mutation operators (our proposed EM-based one against other SM, RM, and CIM), based on 12 different TSPLIB datasets used. The shortest path is highlighted in bold font, which is the best one in solving the TSP problem. Our proposed EM-based model outperformed other mutation strategies in 8 out of 12 datasets, and further, it achieved the second-best average performance in the remaining 4 datasets. The results also show high consistency across runs, indicating the stability of our proposed EM-based framework.

In contrast, the CIM mutation operator did not outperform any other method in any of the datasets, highlighting its limited effectiveness in the tested TSP scenarios. The RM mutation operator showed competitive performance in a few cases, but was generally outpaced by EM and SM in both convergence speed and solution quality.

Another notable observation is the consistent tendency of our proposed EM-based technique to outperform other mutation techniques as the size of the test cases increases. This trend can be attributed to the inherent design of EM, which effectively identifies and preserves essential edges, allowing the algorithm to maintain the structural integrity of high-quality solutions while introducing beneficial diversity. As the problem size and complexity grow, this balance between exploration and exploitation becomes increasingly critical, giving the proposed EM framework a distinct advantage in navigating larger and more complex solution spaces.

Moreover, the ability of our proposed EM-based model to focus mutations on non-essential edges reduces the risk of disrupting critical solution components, leading to more stable and efficient convergence in high-dimensional

Table 4 Comparison of proposed EM model against SM, RM, and CIM mutation operators using the TSPLIB dataset in terms of average execution time

	EM	SM	RM	CIM
Test	Avg Execution Time			
a280	1911.069	1592.275	1515.351	1578.453
berlin52	716.1476	644.6814	607.5467	660.977
bier127	999.3656	851.9747	828.3574	863.7678
ch150	1114.769	933.1215	899.1908	938.6177
eil51	751.9011	633.2922	600.6457	652.716
kroA100	1012.738	809.4399	756.2927	830.7229
kroA200	1439.262	1263.091	1233.515	1242.922
lin318	2211.743	1805.128	1647.211	1807.688
pr152	1248.542	1010.26	1019.447	1006.388
pr226	1517.34	1309.274	1197.858	1313.775
rat195	1277.923	1219.221	1192.932	1207.482
st70	776.9849	683.3245	676.3672	685.6645

problem instances. This scalability positions EM as a robust and adaptive mutation operator with significant potential for solving optimization problems in larger, more complex search spaces, making it an ideal choice for applications that demand high computational efficiency and solution quality.

In several TSP instances—such as a280, ch150, rat195, and st70—our proposed EM-based model achieves competitive or lower STDEV values compared to other SM, RM, and CIM mutation operators. For example, in ch150, the proposed EM model records a STDEV of 1006.24, nearly identical to SM (1002.26), but with a notably better average performance. In the rat195 dataset, the STDEV of the proposed EM model is 313.84, which is lower than the SM model (393.10) and CIM model (368.51), indicating improved consistency while achieving the best average result. These results suggest that our proposed EM-based model not only improves solution quality but also does so reliably across multiple runs, pointing to effective guidance in the search space without excessive disruption of high-quality components.

In contrast, techniques such as RM and CIM exhibit highly variable behavior, with high STDEV values in large instances like pr226 and bier127, indicating less stable convergence. This further confirms that the essential edge preservation strategy in our proposed EM-based model leads to reduced variance, reflecting a better balance between exploration and exploitation. The dynamic essential edge mechanism appears to help prevent premature convergence while retaining high-performing structural components.

3.2 Execution time analysis

Table 4 summarizes the results obtained from the experiments, which reported the average execution time in comparison with our proposed EM model against other SM, RM,

and CIM mutation operators, based on using the TSPLIB datasets. The best execution time is highlighted in bold font style. Remarkably, the RM model has outperformed the others with the shortest execution time for 11 out of 12 datasets. The CIM model has also shown competitive performance in securing the shortest execution time only in one dataset. On the contrary, the proposed EM framework demonstrates a slightly higher average execution time while showing its promising capability.

The higher execution time observed for the proposed EM model can be attributed to its distinct operational characteristics compared to other mutation techniques. Unlike RM and CIM mutation operators, the proposed EM model focuses on identifying and preserving critical edges that should remain unaffected during the mutation process. This additional step, which involves analyzing solution quality across multiple generations, contributes to the increased computational overhead.

However, it is important to emphasize that this added complexity is a deliberate trade-off aimed at enhancing solution quality. By preserving essential edges, the proposed EM model maintains the structural integrity of high-performing solutions while introducing beneficial diversity. This approach, although slightly more time-consuming, ultimately improves the algorithm's ability to converge toward optimal solutions, making it a valuable choice for scenarios where solution quality is prioritized over execution speed.

4 Conclusion and future work

This study adapts the EM technique, which is a novel mutation operator designed to enhance the performance of GAs for the TSP problem. The core contribution lies in EM's unique ability to identify and preserve essential edges—routes that frequently appear in high-fitness tours—thereby maintaining solution integrity while allowing for effective exploration of the solution space. Through extensive experimentation on twelve different benchmark TSP instances from TSPLIB, the proposed EM-based model consistently outperformed traditional mutation operators such as SM, RM, and CIM mutation operators in terms of both solution quality and convergence stability, particularly as the problem size increased.

Although our proposed EM-based model incurred slightly higher computational costs compared to its counterparts, the trade-off yielded significant improvements in the quality of the final solutions. The results clearly demonstrate the proposed EM's potential as a scalable and robust mutation strategy for large and complex optimization problems, where preserving the structural integrity of high-performing solutions is critical.

To strengthen the empirical evaluation, a Wilcoxon signed-rank test was added to verify the statistical significance of the observed performance gains, revealing that the proposed EM-based model significantly outperforms RM and CIM at the 0.05 level. These results support the effectiveness of the proposed EM model.

Future work will extend the scope of this research in several directions. First, we plan to generalize the concept of the EM mutation technique for broader classes of combinatorial optimization problems, such as the Vehicle Routing Problem (VRP), Quadratic Assignment Problem (QAP), and Job-Shop Scheduling. Second, incorporating adaptive mechanisms to dynamically adjust the essential rate based on population diversity and convergence trends could improve both the efficiency and responsiveness of the algorithm. Additionally, hybridizing EM with other metaheuristic strategies – such as Ant Colony Optimization or Particle Swarm Optimization – offers a promising path for further performance enhancement. Third, we aim to benchmark EM against more competitive and modern metaheuristic approaches, including Ant Colony Optimization, Particle Swarm Optimization, and hybrid GA variants, to comprehensively evaluate its comparative performance.

Finally, we aim to investigate parallel and distributed implementations of EM to mitigate its computational overhead, enabling its application in real-time and large-scale optimization scenarios. This research thus lays the groundwork for developing mutation operators that are both structure-aware and performance-optimized, contributing a novel dimension to the design of intelligent evolutionary algorithms.

Author Contributions The first author developed the Essential Mutation framework and drafted the main manuscript text. The second author contributed to the algorithm design and assisted in manuscript writing. The third author implemented the algorithm, conducted the experiments, and prepared Figures 1–3. The fourth author contributed to the analysis, interpretation of results, and manuscript refinement. All authors reviewed and approved the final version of the manuscript.

Data Availability The authors declare that this research was conducted independently and ethically. No conflicts of interest are associated with this study. All experiments were carried out using publicly available benchmark datasets from the TSPLIB repository, which can be accessed at <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>. The MATLAB implementation of the proposed Essential Mutation framework and all supporting code used in the experimental evaluation are available from the corresponding author upon reasonable request. No proprietary data or materials were used in this study.

Declarations

Competing interests The authors declare no competing interests.

Ethics declaration Not applicable.

References

1. Zhang, P., Wang, J., Tian, Z., Sun, S., Li, J., Yang, J.: A genetic algorithm with jumping gene and heuristic operators for traveling salesman problem. *Applied Soft Computing* **127**, 109339 (2022). <https://doi.org/10.1016/j.asoc.2022.109339>
2. Goel, L., Vaishnav, G., Ramola, S.C., Purohit, T.: A modified ant colony optimization algorithm with pheromone mutations for dynamic travelling salesman problem. *IETE Technical Review* **40**(6), 767–782 (2023)
3. Li, J., Liu, Q., Wang, C., Li, X.: A disjunctive graph-based metaheuristic for flexible job-shop scheduling problems considering fixture shortages in customized manufacturing systems. *Robotics and Computer-Integrated Manufacturing* **95**, 102981 (2025)
4. Spears, W.M.: Crossover or mutation? In: *Foundations of Genetic Algorithms vol. 2*, pp. 221–237. Elsevier (1993)
5. Al-Khatib, R.M., Al-Betar, M.A., Awadallah, M.A., Nahar, K.M., Shquier, M.M.A., Manasrah, A.M., Doumi, A.B.: Mga-tsp: modernised genetic algorithm for the travelling salesman problem. *International Journal of Reasoning-based Intelligent Systems* **11**(3), 215–226 (2019)
6. Li, J., Liu, Q., Li, X., Gao, L.: An efficient problem-specific evolutionary algorithm for flexible job shop scheduling problem with specific workers in highly customised manufacturing systems. *International Journal of Production Research* (2025). <https://doi.org/10.1080/00207543.2025.2496971>
7. Doush, I.A., Sawalha, A.: Automatic music composition using genetic algorithm and artificial neural networks. *Malays. J. Comput. Sci.* **33**(1), 35–51 (2020)
8. Abu Doush, I., Al-Saleh, M.I.: Can genetic algorithms help virus writers reshape their creations and avoid detection? *Journal of Experimental & Theoretical Artificial Intelligence* **29**(6), 1297–1310 (2017)
9. Blum, C., Roli, A.: Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys* **35**(3), 268–308 (2003)
10. Alkhateeb, F., Al-Khatib, R.M., Doush, I.A.: A survey for recent applications and variants of nature-inspired immune search algorithm. *Int. J. Comput. Appl. Technol.* **63**(4), 354–370 (2020)
11. Holland, J.H.: Genetic algorithms. *Scientific american* **267**(1), 66–73 (1992)
12. Koza, J.R.: Genetic programming: On the programming of computers. Means of natural selection (1992)
13. Beyer, H.-G., Schwefel, H.-P.: Evolution strategies—a comprehensive introduction. *Natural computing* **1**(1), 3–52 (2002)
14. Yao, X., Liu, Y., Lin, G.: Evolutionary programming made faster. *IEEE Trans. Evol. Comput.* **3**(2), 82–102 (1999)
15. Ramos-Figueroa, O., Quiroz-Castellanos, M., Mezura-Montes, E., Kharel, R.: Variation operators for grouping genetic algorithms: A review. *Swarm and Evolutionary Computation* **60**, 100796 (2021)
16. Ramos-Figueroa, O., Quiroz-Castellanos, M., Mezura-Montes, E., Schütze, O.: Metaheuristics to solve grouping problems: A review and a case study. *Swarm and Evolutionary Computation* **53**, 100643 (2020)
17. Skinderowicz, R.: Improving ant colony optimization efficiency for solving large tsp instances. *Applied Soft Computing* **120**, 108653 (2022)
18. Emambocus, B.A.S., Jasser, M.B., Hamzah, M., Mustapha, A., Amphawan, A.: An enhanced swap sequence-based particle swarm optimization algorithm to solve tsp. *Ieee Access* **9**, 164820–164836 (2021)
19. Jiang, Y., Weise, T., Lässig, J., Chiong, R., Athauda, R.: Comparing a hybrid branch and bound algorithm with evolutionary computation methods, local search and their hybrids on the tsp. In:

- 2014 IEEE Symposium on Computational Intelligence in Production and Logistics Systems (CIPLS), pp. 148–155 (2014). IEEE
20. Monroe, J., Srikant, T., Carbonell-Bejerano, P., Becker, C., Lensink, M., Exposito-Alonso, M., Klein, M., Hildebrandt, J., Neumann, M., Kliebenstein, D., et al.: Mutation bias reflects natural selection in *arabidopsis thaliana*. *Nature* (2022). <https://doi.org/10.1038/s41586-021-04269-6>
 21. Doush, I.A., Awadallah, M.A., Al-Betar, M.A.: A novel essential mutation method for evolutionary algorithms. In: 2022 2nd International Conference on Computing and Machine Intelligence (ICMI), pp. 1–5 (2022). IEEE
 22. Bye, R.T., Gribbestad, M., Chandra, R., Osen, O.L.: A comparison of ga crossover and mutation methods for the traveling salesman problem. In: *Innovations in Computational Intelligence and Computer Vision: Proceedings of ICICV 2020*, pp. 529–542 (2021). Springer
 23. Reinelt, G.: TSPLIB. University of Heidelberg. <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/> (1996)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.